개발일지 — EPL (Easy Pick League) Phase 1

항목	내용
날짜	2026년 4월 24일 (금)
프로젝트	EPL (Easy Pick League) — 아마추어 축구 경기 자동 매칭 플랫폼
담당자	2205 김태균
개발 환경	React 19.x / Vite 8.x / LocalStorage / macOS

1. 학습한 내용 및 활동 결과

1-1. 개발 환경 구성

Vite + React 19 기반의 프론트엔드 프로젝트 초기 환경을 구성하였다. 빌드 도구로 Vite를 선택한 이유는 HMR(Hot Module Replacement) 속도가 webpack 대비 현저히 빠르고, ES Module 기반의 네이티브 번들링으로 개발 서버 구동 속도를 최소화할 수 있기 때문이다. 서버 없이 LocalStorage만으로 데이터를 관리하는 구조를 채택하여 MVP 개발 속도를 높이고 배포 복잡도를 낮추는 전략을 적용하였다.

1-2. 공통 UI 컴포넌트 설계 및 구현

재사용 가능한 공통 컴포넌트 6종을 설계하고 구현하였다. CSS 변수(Custom Properties) 기반 디자인 토큰을 도입하여 색상, 간격, 폰트 크기 등 시각적 일관성을 코드 수준에서 강제하는 구조를 만들었다. 컴포넌트 계층은 `Layout > Container > Card` 순으로 감싸며, `Button`, `Input`, `Header`는 독립적으로 조합 가능하도록 설계하였다.

> **구현 컴포넌트 목록**: Button, Card, Input, Header, Layout, Container

1-3. Phase 1 핵심 기능 구현

경기 모집 및 참여 취합 기능(Phase 1)의 핵심 구성 요소를 모두 구현하였다.

구현 내용 요약

구성 요소	파일	기능
경기 생성 폼	`MatchForm.jsx`	경기명·장소·일시·정원·설명 입력, 클라이언트 사이드 유효성 검사
참여자 등록 폼	`ParticipantForm.jsx`	닉네임·학년·포지션 입력, 정원 초과 시 입력 잠금
참여자 목록	`ParticipantList.jsx`	등록 참여자 카드 렌더링, 개별 제거 기능
경기 관리 액션	`MatchActions.jsx`	경기 상태 전환(모집중 → 준비완료), 삭제 버튼
데이터 훅	`useMatchData.js`	경기 CRUD + 참여자 CRUD, LocalStorage 연동
폼 상태 훅	`useForm.js`	다중 폼 재사용 가능한 범용 폼 상태 관리
CSV 내보내기	`csvExport.js`	한글 BOM 처리 포함 CSV 다운로드 유틸
라우팅	`MatchPage.jsx`	URLSearchParams 기반 경기 ID 라우팅

소스코드 캡처 ①: `useLocalStorage.js` — LocalStorage와 React State 동기화 훅

```
// src/hooks/useLocalStorage.js
function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      if (item) {
        return JSON.parse(item);
      }
      return initialValue;
    } catch (error) {
      console.error(`LocalStorage 접근 오류 (${key}):`, error);
      return initialValue;
    }
  });

  const setValue = (value) => {
    try {
      // 함수가 전달된 경우 실행 (prev => next 패턴 지원)
      const valueToStore = value instanceof Function ? value(storedValue) :
value;
      setStoredValue(valueToStore);
      window.localStorage.setItem(key, JSON.stringify(valueToStore));
    } catch (error) {
      console.error(`LocalStorage 저장 오류 (${key}):`, error);
    }
  };

  return [storedValue, setValue];
}
```

> **학습 포인트**: `useState`의 lazy initializer(함수 형태 초기값)를 활용하여 컴포넌트 마운트 시 단 한 번만 LocalStorage를 읽도록 최적화하였다. 또한 `setValue`에서 함수 인자를 허용함으로써 `setMatches((prev) => [...prev, newItem])` 패턴이 안전하게 동작하도록 설계하였다.

소스코드 캡처 ②: `useMatchData.js` — 경기 및 참여자 CRUD 훅 (핵심 로직)

```
// src/hooks/useMatchData.js
function useMatchData() {
  const [matches, setMatches] = useLocalStorage('epl_matches', []);

  // 새 경기 생성 — timestamp 기반 고유 ID 자동 부여
  const createMatch = useCallback(
    (matchData) => {
      const newMatch = {
        id: `match_${Date.now()}`,
        createdAt: new Date().toISOString(),
        status: 'recruiting',
```

```
        participants: [],
        ...matchData,
      };
      setMatches((prev) => [newMatch, ...prev]);
      return newMatch;
    },
    [setMatches]
  );

  // 참여자 추가 — 경기 배열 내 불변 업데이트
  const addParticipant = useCallback(
    (matchId, participantData) => {
      const participant = {
        id: `p_${Date.now()}`,
        joinedAt: new Date().toISOString(),
        ...participantData,
      };
      setMatches((prev) =>
        prev.map((match) =>
          match.id === matchId
            ? { ...match, participants: [...match.participants,
participant] }
            : match
        )
      );
      return participant;
    },
    [setMatches]
  );

  return { matches, createMatch, addParticipant, /* ... */ };
}
```

> **학습 포인트**: 배열 내 중첩 객체를 불변(immutable)하게 업데이트하기 위해 `map` + spread 패턴을 사용하였다. `useCallback`으로 함수 참조를 메모이제이션하여 하위 컴포넌트의 불필요한 리렌더링을 방지하였다.

소스코드 캡처 ③: `csvExport.js` — 한글 BOM 처리 CSV 내보내기 유틸

```
// src/utils/csvExport.js

// CSV 값 이스케이프 (쉼표·줄바꿈·큰따옴표 포함 시 큰따옴표로 감싸기)
const escapeCSV = (value) => {
  if (value === null || value === undefined) return '';
  const stringValue = String(value);
  if (stringValue.includes(',') || stringValue.includes('\n') ||
stringValue.includes('"')) {
    return `"${stringValue.replace(/"/g, '""')}"`;
  }
  return stringValue;
};
```

```
// BOM 추가로 Excel 한글 깨짐 방지
export const downloadCSV = (csvContent, filename = 'export.csv') => {
  const BOM = '';
  const blob = new Blob([BOM + csvContent], { type: 'text/csv;charset=utf-
8;' });
  const link = document.createElement('a');
  const url = URL.createObjectURL(blob);
  link.setAttribute('href', url);
  link.setAttribute('download', filename);
  document.body.appendChild(link);
  link.click();
  document.body.removeChild(link);
  URL.revokeObjectURL(url);  // 메모리 누수 방지
};
```

> **학습 포인트**: Windows Excel에서 UTF-8 CSV 파일의 한글이 깨지는 문제는 BOM()을 파일 앞에 삽입하여 해결할 수 있다. `URL.revokeObjectURL`로 Blob URL을 즉시 해제하여 메모리 누수를 방지하는 패턴을 적용하였다.

소스코드 캡처 ④: `ParticipantForm.jsx` — 정원 초과 제어 로직

```
// src/components/ParticipantForm.jsx (핵심 로직 발췌)
function ParticipantForm({ onSubmit, currentCount = 0, maxCount = 20,
loading = false }) {
  const isFull = currentCount >= maxCount;

  const handleSubmit = (e) => {
    e.preventDefault();
    if (isFull) {
      alert('참가 인원이 가득 찼습니다.');
      return;
    }
    const formErrors = validateForm();
    if (Object.keys(formErrors).length > 0) {
      Object.entries(formErrors).forEach(([field, error]) => {
        form.setFieldError(field, error);
      });
      return;
    }
    form.handleSubmit(e);
    form.reset();  // 성공 후 폼 즉시 초기화
  };

  return (
    <Card header={<h3>👤 참여 등록</h3>}>
      {/* ... */}
      <Button variant="primary" disabled={isFull}>
        {isFull ? '참가 불가' : '참여하기'}
      </Button>
      {isFull && (
```

```
          <div className="warning-box">⚠️ 참가 인원이 모두 찼습니다.</div>
        )}
      </Card>
    );
  }
```

1-4. 빌드 검증

`npm run build` 실행 결과 Vite 빌드 성공을 확인하였다. 초기 빌드 시 발생한 두 건의 import 경로 오류를 수정한 후 에러 없이 `dist/` 디렉토리에 번들 파일이 생성되었다.

---

2. 고찰 및 추후 개발 계획

2-1. 고찰

이번 Phase 1 구현을 통해 컴포넌트 간 데이터 흐름과 LocalStorage 동기화의 구조적 패턴을 확립하였다. 특히 `useLocalStorage → useMatchData → Page` 계층 구조는 데이터 접근 책임을 명확히 분리해 가독성을 높였다. 반면, 참여자 추가 후 `currentMatch`를 `matches` 배열에서 다시 조회하는 패턴은 LocalStorage 쓰기와 React 상태 갱신 사이의 타이밍 차이로 인해 동기화 불안정 가능성이 존재하며, 이는 `useEffect` 의존성 정리 또는 직접 반환값 활용으로 개선할 필요가 있다. 또한 `handleDeleteMatch`가 `deleteMatch`를 실제 호출하지 않아 LocalStorage 데이터가 남는 미완성 이슈가 남아있어 다음 세션에서 즉시 처리할 예정이다.

2-2. 추후 개발 계획

우선순위	작업	Phase
긴급	`handleDeleteMatch`에 `deleteMatch` 실제 호출 연결	Phase 1 마무리
높음	참여자 추가 후 `currentMatch` 동기화 로직 안정화	Phase 1 마무리
높음	Snake Draft 팀 빌딩 알고리즘(`src/utils/snakeDraft.js`) 구현	Phase 2 시작
중간	학년 가중치 기반 팀 편향 방지 필터(`yearWeighting.js`) 설계	Phase 2
중간	팀 배정 결과 UI 및 재배정 버튼 컴포넌트 구현	Phase 2
낮음	인라인 스타일을 CSS 파일로 분리, 폼 검증 로직 공통화	리팩토링

---

3. 이슈 발생 및 처리

이슈 #1 — [해결] Vite import-analysis 경로 오류

항목	내용
발생 위치	`src/pages/MatchPage.jsx`
증상	Vite 빌드 시 `Failed to resolve import "./components/..."` 오류 발생
상태	✅ 해결 완료

원인 분석 및 해결 과정

파일을 `src/` 루트에서 `src/pages/` 하위로 이동한 이후, import 경로를 갱신하지 않아 발생한 오류이다. Vite는 ES Module 사양을 엄격하게 준수하므로 상대 경로가 실제 파일 위치를 정확히 가리켜야 한다. `src/pages/MatchPage.jsx`는 `src/components/`, `src/hooks/` 폴더와 한 단계 위 디렉토리(부모)를 공유하므로 `./components` → `../components`, `./hooks` → `../hooks`로 경로를 수정하는 것이 올바른 처리이다. 이를 통해 모듈 해석이 정상화되어 빌드가 성공적으로 완료되었다. 향후 파일 이동 시에는 IDE의 자동 import 경로 갱신 기능을 활용하거나 절대 경로 alias(`@/components`)를 `vite.config.js`에 설정하는 것이 근본적인 예방책이 될 수 있다.

```
// 수정 전 (오류)
import { Layout, MatchForm } from './components';
import { useMatchData } from './hooks';

// 수정 후 (정상)
import { Layout, MatchForm } from '../components';
import { useMatchData } from '../hooks';
```

이슈 #2 — [해결] `useForm` 잘못된 import 경로로 인한 빌드 오류

항목	내용
발생 위치	`src/components/MatchForm.jsx`, `src/components/ParticipantForm.jsx`
증상	`useForm is not exported by src/utils/index.js` 빌드 에러 발생
상태	✅ 해결 완료

원인 분석 및 해결 과정

`useForm`은 `src/hooks/` 디렉토리에 정의된 커스텀 훅임에도 불구하고 컴포넌트에서 `../utils`로 잘못 import하여 발생한 오류이다. React 프로젝트에서 커스텀 훅(use- 접두사)은 반드시 `hooks/` 디렉토리에 위치해야 한다는 컨벤션을 지키지 않은 것이 직접 원인이 아니라, 경로 입력 실수가 원인이었다. `src/utils/index.js`에는 `csvExport`, `validators`, `constants`만 re-export되므로 `useForm`이 존재하지 않아 번들러가 오류를 발생시킨 것이다. `../utils` → `../hooks`로 경로를 수정한 후 빌드가 정상화되었다.

```
// 수정 전 (오류)
import { useForm } from '../utils';

// 수정 후 (정상)
import { useForm } from '../hooks';
```

> 프로젝트 규모가 커질수록 각 디렉토리의 `index.js`에서 명확하게 re-export를 관리하고, 각 디렉토리의 역할(utils = 순수 함수, hooks = React 훅)을 엄격히 구분하는 것이 유사 오류를 예방하는 핵심 방법이다.

이슈 #3 — [미해결] `handleDeleteMatch` 실제 데이터 삭제 미연결

항목	내용
발생 위치	`src/pages/MatchPage.jsx` — `handleDeleteMatch` 함수
증상	경기 삭제 버튼 클릭 시 화면 상태(`currentMatch`)만 초기화되고, LocalStorage의 경기 데이터는 그대로 남음
상태	⚠️ 미해결 — 다음 세션에서 처리 예정

원인 분석

`useMatchData` 훅에서 `deleteMatch` 함수를 정상적으로 export하고 있음에도 `MatchPage`의 구조 분해 할당에서 `deleteMatch`를 포함하지 않았고, `handleDeleteMatch` 내부에서도 호출하지 않았다. 현재는 `setCurrentMatch(null)`로 화면에서만 경기를 숨기는 동작을 하고 있어, 페이지 새로고침 시 해당 경기가 다시 목록에 나타나는 문제가 발생한다.

조치 예정

```
// 수정 예정 코드
const { matches, createMatch, addParticipant, removeParticipant,
updateMatch, deleteMatch } = useMatchData();

const handleDeleteMatch = () => {
  if (!window.confirm('경기를 삭제하시겠습니까?')) return;
  deleteMatch(currentMatch.id);          // LocalStorage에서도 제거
  setCurrentMatch(null);
  window.history.pushState(null, '', '/');
};
```

---

4. 오늘의 작업 요약

```
Phase 1 핵심 기능 구현 완료 (안정화 2건 미완)

✅ 완료:
  - React + Vite 환경 구성
  - 공통 UI 컴포넌트 6종 (Button, Card, Input, Header, Layout, Container)
  - 경기 생성 폼 (MatchForm)
  - 참여자 등록 폼 (ParticipantForm)
  - 참여자 목록 (ParticipantList)
  - LocalStorage CRUD 훅 (useMatchData, useLocalStorage)
  - 범용 폼 상태 훅 (useForm)
  - CSV 내보내기 유틸 (csvExport)
  - URLSearchParams 라우팅 (MatchPage)
  - import 경로 오류 2건 해결 및 빌드 성공

⚠️ 미완료:
  - handleDeleteMatch ↔ deleteMatch 연결
  - addParticipant 후 currentMatch 동기화 개선
```

```
🎯 다음 목표:
  - Phase 1 마무리 (위 2건 처리)
  - Phase 2 Snake Draft 알고리즘 설계 착수
```

---

5. 추가 진행 사항 (Phase 1 안정화 1차)

5-1. 삭제 기능 실제 데이터 연동 완료

- 위치: src/pages/MatchPage.jsx
- 조치:
  - useMatchData 구조 분해에 deleteMatch 추가
  - handleDeleteMatch에서 deleteMatch(currentMatch.id) 호출
  - 삭제 후 selectedMatchId/currentMatch/URL 동기화
- 결과: 화면에서만 사라지는 문제가 해소되고 LocalStorage에서도 실제 삭제됨

5-2. currentMatch 동기화 구조 개선

- 위치: src/pages/MatchPage.jsx
- 조치:
  - selectedMatchId 상태 추가
  - matches 또는 selectedMatchId 변경 시 useEffect로 currentMatch 재계산
  - add/remove/status 핸들러 내부 수동 setCurrentMatch 제거
- 결과: 참가자 추가/삭제 직후 발생 가능한 stale 상태 문제 예방

5-3. 코드 안정성 개선

- parseInt 호출에 radix(10) 명시
- null 가드 추가로 예외 호출 차단

5-4. 검증

- 명령: npm run build
- 결과: 성공 (vite build 통과)

5-5. 상태 업데이트

- 기존 미완료 2건(삭제 연동, 동기화 개선) 해결 완료
- Phase 1 잔여 항목은 홈/경기 페이지 전환 로직 정리

---

6. 추가 진행 사항 (페이지 전환 로직 정리)

6-1. URL 기반 페이지 동기화 강화

- 위치: src/App.jsx
- 조치:
  - page 쿼리 파라미터(page=match) 인식 로직 추가
  - matchId 존재 시 경기 페이지 우선 진입 유지

    - popstate 이벤트 등록으로 브라우저 뒤로가기/앞으로가기 동기화
- 결과: 홈/경기 전환 시 URL과 화면 상태의 일관성 개선

6-2. 네비게이션 링크 체계 통일

- 위치: src/pages/MatchPage.jsx
- 조치:
    - /match, /teams, /reviews 링크를 쿼리 기반 링크로 통일
    - 경기 링크: /?page=match
    - 팀 구성 링크: /?page=teams
    - 평가 링크: /?page=reviews
- 결과: 현재 단순 라우팅 구조와 링크 체계 충돌 해소

6-3. 검증

- 명령: npm run build
- 결과: 성공 (vite build 통과)

6-4. 마감 상태

- Phase 1 안정화 포함 완료
- 다음 작업 시작점: Phase 2 Snake Draft 팀 빌딩 알고리즘 구현

---

7. 추가 진행 사항 (Phase 2 시작)

7-1. 알고리즘 유틸 구현

- 위치:
    - src/utils/snakeDraft.js
    - src/utils/yearWeighting.js
- 구현 내용:
    - Snake Draft 패턴(A-B-B-A) 기반 팀 분배
    - 실력 차이 + 학년 페널티를 합산한 밸런스 점수 계산
    - 반복 시뮬레이션으로 가장 균형 점수가 낮은 조합 선택

7-2. 팀 구성 화면 구현

- 위치:
    - src/pages/TeamPage.jsx
    - src/components/TeamBuilder.jsx
    - src/styles/TeamBuilder.css
- 구현 내용:
    - 경기 선택 드롭다운
    - 학년 가중치/반복 횟수 슬라이더
    - 재배정 버튼
    - 팀 A/B 결과 카드 및 밸런스 지표 표시

7-3. 참가자 데이터 확장

- 위치:
  - src/components/ParticipantForm.jsx
  - src/pages/MatchPage.jsx
  - src/components/MatchActions.jsx
- 구현 내용:
  - 참여 등록 폼에 실력 점수(1~5) 필드 추가
  - 참여자 저장 시 skillScore 포함
  - CSV 내보내기에 실력점수 컬럼 반영

7-4. 라우팅 연동

- 위치: src/App.jsx
- 구현 내용:
  - page=teams 라우팅 처리
  - TeamPage 렌더 분기 추가

7-5. 검증

- 명령: npm run build
- 결과: 성공 (vite build 통과)